AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

**Type 1: Research Project**

**Project Title:** Addressing optimization issues in game design using evolutionary algorithms
**Project Scope:** Strategy and Parameter Optimization of games with co-evolving competitive game agents using evolutionary algorithms.

**Team Number: 99**

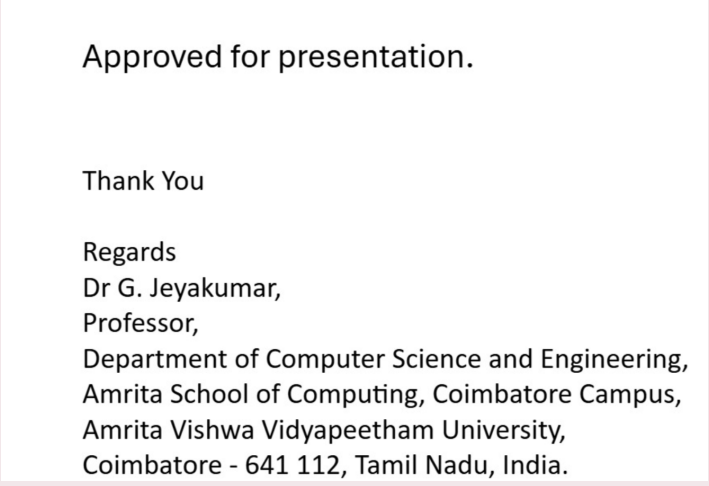
| S.No | Register Number | Student Name |
|---|---|---|
| 1 | CB.SC.U4CSE23152 | Vinaayak Kanagaraj |
| 2 | CB.SC.U4CSE23215 | Darun R G |
| 3 | CB.SC.U4CSE23232 | Pradeep S |
| 4 | CB.SC.U4CSE23235 | Anusanth R |
| 5 | CB.SC.U4CSE23256 | Sathya Roopan M |

Guide: **Dr Jeyakumar G**
Professor, Department of CSE

March 30, 2026

# Approval

## Approval Photo


Approved for presentation.

Thank You

Regards
Dr G. Jeyakumar,
Professor,
Department of Computer Science and Engineering,
Amrita School of Computing, Coimbatore Campus,
Amrita Vishwa Vidyapeetham University,
Coimbatore - 641 112, Tamil Nadu, India.

# Problem Statement

- **Problem Statement:** Strategy Optimization and Co-Evolution of Competitive Game Agents Using Evolutionary Algorithms

- **Abstract:**This work introduces a game system where agents learn and improve continuously, rather than being trained once for fixed goals. Agents are evaluated using weighted performance histories across multiple generations — capturing adaptive capability over time rather than momentary performance, while operating within a fixed reward structure that ensures consistent and reproducible fitness measurement. Instead of adapting to opponent behavior alone, agents evolve in response to shifts in the game environment itself — changes in map configuration, resource distribution, and competitive conditions across generations.

  A diversity-preservation mechanism prevents populations from converging to similar behaviors, while a Hall of Fame archive ensures agents are tested against historically strong opponents rather than only current ones. Together, these mechanisms aim to produce game agents that remain competitive, strategically diverse, and stable as the environment evolves — with all key components formally defined and validated on the microRTS simulation environment.

# Comments from the First Review

| Reviewer Comment | How We Addressed It |
|---|---|
| Explore ways to validate the system | By implementing 3 baseline algorithms (GA, ES, Hill Climbing) for comparative benchmarking across 8×8 and 16×16 maps. [1] |
| Evolutionary algorithms are resource-heavy — may need 2–4 GPUs | Since we use a lightweight game (microRTS) with a compact 6-parameter search space, the entire training runs on a single CPU in under 60 seconds with no GPU needed. |
| Try simpler games like boundary or territorial games first | Adopted microRTS — a lightweight, deterministic real-time strategy engine specifically designed for AI research, with scalable map sizes (8×8 to 32×32). |
| Reduce parameter space — too many parameters adds complexity | Reduced the evolved parameter space to a compact 6-dimensional DNA vector, one weight per bot strategy type. |
| Clarify the target game — consider whether a full RTS is realistic | Scoped to microRTS rather than a commercial RTS which helped to finish the game execution in shorter time.[1] |
| Update references — use recent works in Computational Intelligence | Updated with recent references from 2018 to 2025. |

# Literature Survey Summary

- Evolutionary algorithms are widely used in game research due to their ability to explore large search spaces and generate high-quality solutions for agents, content, and game balancing.
- Prior work demonstrates applications of EAs in:
  - Game balancing and disruption in RTS environments
  - Dynamic difficulty adjustment and adaptive gameplay
  - Game economy modeling and parameter tuning
  - Procedural content generation such as enemies and levels [2]
- EAs have also been successfully used for:
  - Optimizing RTS bot strategies
  - Discovering complex action sequences in games
  - Evolving cooperative and competitive agent behaviors

# Literature Survey Summary

- Evolutionary computation supports both:
  - Gameplay optimization (agent strategies, performance)
  - Content generation (levels, enemies, economies)
- Existing works focus on different optimization goals:
  - Agent performance and strategy optimization
  - Game difficulty control and adaptation
  - Multi-agent competitive and cooperative learning .
  - Balancing complex systems such as game economies
- Co-evolution plays a key role in multi-agent systems:
  - Agents evolve in response to each other
  - Leads to adaptive strategies and emergent behaviors .

# Research Gaps Identified

- Most existing approaches are **game-specific**, limiting generalization across different environments.
- Many methods rely on **manual tuning**, hand-crafted rules, or fixed parameter settings, reducing scalability and adaptability.
- Current approaches often optimize **single objectives** (e.g., win rate, difficulty), without considering long-term adaptation or evolving opponent behavior.
- Limited focus on **time-dependent performance** and strategy evolution across multiple generations.
- There is a need for more **adaptive, trajectory-aware, and coevolutionary frameworks** that can handle dynamic and complex game environments effectively.

# Challenges in Implementation

## 1. Computational Cost of Fitness Evaluation

- Every generation requires each agent to play full game simulations against opponents — total rollouts grow quickly with population size.
- **Our approach:** N=20 agents, K=2 games each, run in headless mode via `BatchGameRunner` — simulating a full game in milliseconds, making 100 generations feasible on a single CPU.[1]

## 2. Defining a Stable Fitness Measure

- Single-game win rate is noisy — one bad game can discard a genuinely good strategy. Standard GA only looks at the most recent result.
- **Our approach:** Trajectory score $e_i$ — weighted average of the last 5 generations, where past scores are weighted by how similar the past environment was to the current one (Euclidean distance on population centroids).

## 3. Agents Getting Stuck in Local Optima

- A fixed mutation rate cannot distinguish a genuinely good agent from a lucky one — both explore at the same rate.
- **Our approach:** Adaptive mutation $\mu_i = \mu_{base} \times (1 - e_i) + 0.05$ — agents doing well mutate less, struggling agents mutate more.

# Challenges in Implementation

## 4. Premature Convergence and Loss of Diversity

- Evolutionary pressure drives agents toward similar strategies — once populations converge, improvement stops entirely.
- **Our approach:** Two-layer pruning — weakness pruning removes agents below $\delta_f = 0.25$ after 3+ generations, and redundancy pruning removes agents with DNA similarity above $\delta_s = 0.85$.

## 5. Strategy Representation

- Defining what an agent strategy means in a concrete, evolvable form is non-trivial — too complex and evolution slows, too simple and the space is meaningless.
- **Our approach:** Each agent is a 6-float DNA vector — one value per bot type (WorkerRush, LightRush, HeavyRush, etc.). Active strategy chosen by `argmax` — small, interpretable, and directly evolvable. [1]

## 6. Choosing the Right Validation Environment

- Full RTS games are computationally intractable for evolutionary runs on standard hardware.
- **Our approach:** microRTS — deterministic, headless, and fast enough to complete 4000 total games across 100 generations in under a minute on a single CPU.

# Technical Validation - TD-CCEA

What makes TD-CCEA different from standard evolutionary algorithms - Trajectory-Driven Competitive Co-Evolutionary Algorithm.

## Core Idea

Unlike standard EAs that judge agents only on their latest game, TD-CCEA uses **past performance history** weighted by **environment similarity** to score agents — making evaluation smarter and more stable.

**Key Contributions:**

1. **Trajectory-Based Scoring:** Instead of raw win/loss, agents are scored using a weighted average of their last 5 results, where recent similar environments count more.
2. **Dynamic Environment Awareness:** A population centroid captures the current competitive landscape. Past results are weighted by how closely they match the present environment using Euclidean similarity.
3. **Self-Adapting Mutation:** Agents that perform well mutate slowly (preserving good DNA). Agents that perform poorly mutate aggressively (escaping dead ends). No manual tuning needed.
4. **Diversity Preservation:** Weak agents (below 25% score) are pruned. Near-duplicate agents (above 85% DNA similarity) are removed. This keeps the population diverse and efficient.

## Algorithm

- **We introduce a unified evolutionary framework where fitness history and environment semantics directly shape parameter evolution.**

Initialize population $\mathcal{P}_0$ of $N$ agents, archive $\mathcal{A} \leftarrow \emptyset$;

**for** $t = 1, 2, \ldots, T_{\max}$ **do**

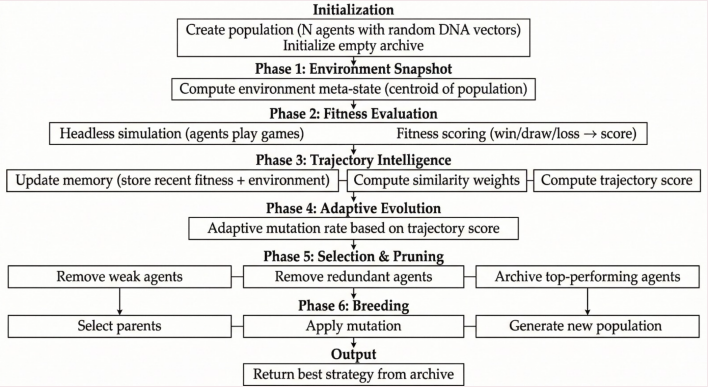
- Compute environment meta-state $\mathbf{c}_t = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{\theta}_i^t$ ;
- Evaluate fitness $f_i^t$ for all $\mathcal{O}_i$ via $K$ simulated games;
- Update trajectory memory $H_i \leftarrow H_i \cup \{(f_i^t,\ \mathbf{c}_t)\}$; trim if $|H_i| > W$;
- Compute dynamic similarity weights $\alpha_k = 1 - \frac{d(\mathbf{c}_t,\ \mathbf{c}_k)}{\hat{d}_{\max}}$ for each memory $k \in H_i$;
- Normalize weights $\hat{\alpha}_k = \frac{\alpha_k}{\sum_j \alpha_j}$ ;
- Compute trajectory score $e_i^t = \sum_k \hat{\alpha}_k \cdot f_i^{(k)}$ ;
- Compute adaptive mutation rate $\mu_i = \mu_{\text{base}} \cdot (1 - e_i^t) + \epsilon$ ;
- Remove $\mathcal{O}_i$ if $\bar{e}_i < \delta_f$ and $|H_i| \geq 3$;
- Remove redundant $\mathcal{O}_j$ if $S(i, j) > \delta_s$ ;
- Insert top-$m$ survivors into $\mathcal{A}$; trim if $|\mathcal{A}| > A_{\max}$;
- Breed offspring from survivors $\cup\ \mathcal{A}$ using Gaussian mutation $\mathcal{N}(0,\ \mu_i)$;

**return** best $\mathcal{O}^* \in \mathcal{A}$;

# Technical Validation - TD-CCEA

## Architecture of our Algorithm

- **This is our TD-CCEA Architecture Diagram. It represents the full iterative workflow of the algorithm: fitness evaluation with trajectory buffering, expressibility scoring, environment-aware hyperparameter adaptation, diversity control, archive maintenance, and reproduction.**



# Technical Validation : Chosen Environments

**microRTS**

- A lightweight, fully deterministic Real-Time Strategy game simulator built for AI research.
- Supports two competing players with units, resources, base-building, and combat in a compact, configurable grid-based world.
- Provides a built-in game engine with API access, fast simulation speed, and straightforward parameter control.[1]

**Diplomacy (No-Press Variant)**

- A multi-player strategy board game (7 players, 75 provinces) where agents compete for territorial control through simultaneous move submission.
- In the No-Press variant, natural language negotiation is removed — agents interact purely through actions, making emergent strategic behavior the sole driver of performance.
- A well-established benchmark environment for multi-agent competitive and cooperative AI research.

# Technical Validation — microRTS

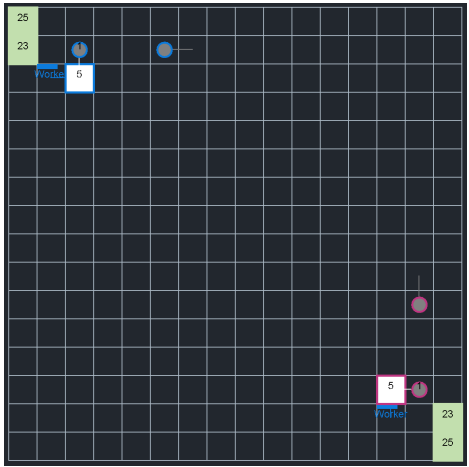
A lightweight Real-Time Strategy simulator built for AI research.

**Game Mechanics:**

- **2 players** compete on a grid-based map
- Victory is achieved by **destroying the opponent's base** or eliminating all units
- Each game tick is a discrete timestep where agents submit **simultaneous unit actions**
- Game parameters — map size, unit stats, resource density, time limits — are **fully configurable**[3]

**Our Configuration:**

- Map sizes: **8**×**8** and **16**×**16** grids
- Starting units: 1 Base + 1 Worker per player
- Starting resources: 5 minerals per player
- Configurable parameters enable evolving unit stats, resource rates, and map layouts alongside agent strategies[1]




*Starting state of the game*

# Why microRTS — Validation & Results

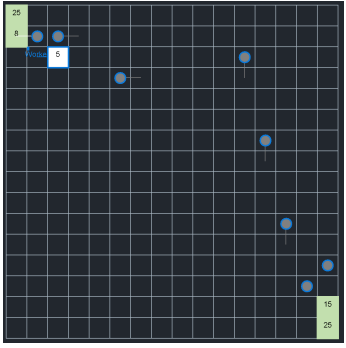
**Why microRTS is ideal for our research:**

- Two-player adversarial structure is a natural fit for the **Predator–Prey CCEA** setup
- Thousands of simulations can be run efficiently — population-based training is feasible **without dedicated GPU clusters**
- Full reproducibility across generations ensures **reliable fitness evaluation**
- Capable of simulating lightweight games for ML validation

**Training Results:**

- TD-CCEA evolved over **100 generations** with a population of 20 agents
- The algorithm autonomously discovered the optimal strategy for each map topology
- Our evolved agent achieved a **100% win rate** against benchmark opponents across 8×8 and 16×16 map environments[1]

*Ending of the game*



# microRTS — Opponent Strategies

Our AI (Player 0, Blue) is evolved using TD-CCEA. It is tested against the following fixed opponent strategies (Player 1, Red):

| **Strategy** | **Behavior** |
|---|---|
| WorkerRush | Sends all workers directly to attack the enemy base as fast as possible |
| LightRush | Builds a barracks, trains Light units, and rushes the opponent |
| HeavyRush | Builds a barracks, trains slow but powerful Heavy units to attack |
| RandomBiased | Makes semi-random decisions with a slight preference for attacking |
| Random | Completely random actions each turn |
| Passive | Does nothing — useful as a baseline |

The AI's 6-dimensional DNA vector has one weight per strategy. The **highest weight** determines which strategy the agent uses in battle.

# Future Work

## Semester 2 — Algorithm Extensions

- **Multi-player scaling:** Extend TD-CCEA to support 10–100 simultaneous competing agents, analysing emergent coalition and arms-race dynamics at scale.
- **Probabilistic search reduction:** Methodologies such as intentional state-violation coupled with backpropagation, inter-sequence compatibility as well as the integration of Markov Models to constrain the evolutionary search space — motivated by DipLLM, which required only 1.5% of the training space of Meta's Cicero model. Combine with mutated exploration and backpropagation of trial search paths [4] [5] .
- **Decentralized fitness computation:** Distribute competition and fitness evaluation across spatial maps using concurrent execution, reducing per-generation wall-clock time for large populations.

## Semester 3 — Framework Generalization

- **Abstracted strategic games:** Adapt the TD-CCEA framework to Diplomacy (binary alliance–betrayal model, no inter-agent messaging), Battleship and Checkers — testing generalization across increasingly complex decision spaces[4] [5].
- **Biological evolution modeling:** Simulate a population evolving through diversified sequential catastrophic environmental shifts, each causing partial collapse and rapid changes in fitness landscapes.

# References I


- J. Snell, M. Masek, and C. P. Lam, “An evolutionary approach to balancing and disrupting real-time strategy games,” in *Proceedings of the 24th International Congress on Modelling and Simulation (MODSIM2021)*, Sydney, NSW, Australia, December 2021, pp. 967–973.
- L. T. Pereira, B. M. F. Viana, and C. F. M. Toledo, “Procedural enemy generation through parallel evolutionary algorithm,” in *2021 20th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, 2021, pp. 126–133.
- D. Novak, D. Verber, J. Dugonik, and I. J. Fister, “A comparison of evolutionary and tree-based approaches for game feature validation in real-time strategy games with a novel metric,” *Evolutionary Computation 2020*, 2020.
- K. Xu, J. Chai, S. Li, Y. Fu, Y. Zhu, and D. Zhao, “Dipllm: Fine-tuning llm for strategic decision-making in diplomacy,” in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

# References II


- T. Anthony, T. Eccles, A. Tacchetti, J. Kramár, I. Gemp, T. C. Hudson, N. Porcel, M. Lanctot, J. Pérolat, R. Everett, R. Werpachowski, S. Singh, T. Graepel, and Y. Bachrach, "Learning to play no-press diplomacy with best response policy iteration," in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- Y. Widhiyasana, M. Harika, F. F. N. Hakim, F. Diani, K. S. Komariah, and D. R. Ramdania, "Genetic algorithm for artificial neural networks in real-time strategy games," *International Journal on Informatics Visualization*, 2022.
- A. Rawal, P. Rajagopalan, and R. Miikkulainen, "Constructing competitive and cooperative agent behavior using coevolution," in *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*. IEEE, 2012, pp. 1–8.
- G. L. Zuin, Y. P. A. Macedo, L. Chaimowicz, and G. L. Pappa, "Discovering combos in fighting games with evolutionary algorithms," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2016)*. Denver, CO, USA: ACM, 2016, pp. 277–284.

# References III


- S. M. Lucas, J. Liu, I. Bravi, R. D. Gaina, J. Woodward, V. Volz, and D. Perez-Liebana, "Efficient evolutionary methods for game agent optimisation: Model-based is best," in *Proceedings of the AAAI Workshop on Games and Simulations for Artificial Intelligence*. AAAI Press, 2019, pp. 1–8.
- H. Armanto, H. A. Rosyid, Muladi, and Gunawan, "Evolutionary algorithm in game – a systematic review," *Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control*, vol. 8, no. 2, pp. 599–606, 2023.
- K. O. Andrade, R. C. Joaquim, G. A. P. Caurin, and M. K. Crocomo, "Evolutionary algorithms for a better gaming experience in rehabilitation robotics," *ACM Computers in Entertainment*, vol. 16, no. 2, pp. 1–15, 2018.
- M. A. Fajardo, P. K. Lehre, and S. Lin, "Runtime analysis of a co-evolutionary algorithm: Overcoming negative drift in maximin-optimisation," in *FOGA '23: Foundations of Genetic Algorithms XVII*, 2023.

# References IV


- D. Perez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game ai: A multitrack framework for evaluating agents, games, and content generation algorithms," *IEEE Transactions on Games*, vol. 11, no. 3, pp. 195–214, 2019.
- A. Fernandez-Ares, A. M. Mora, J. J. Merelo, P. Garcia-Sanchez, and C. Fernandes, "Optimizing player behavior in a real-time strategy game using evolutionary algorithms," in *Proceedings of the 2011 IEEE Congress on Evolutionary Computation (CEC)*. New Orleans, USA: IEEE, 2011, pp. 2017–2024.
- R. D. Gaina, S. Devlin, S. M. Lucas, and D. Perez-Liebana, "Rolling horizon evolutionary algorithms for general video game playing," *IEEE Transactions on Games*, vol. 14, no. 2, pp. 232–242, 2022.
- R. Guha, R. Mckendrick, B. Feest, and K. Deb, "Multi-objective competitive co-evolutionary optimization and regularity-based decision-making for two-agent wargame strategy optimization," *IEEE Transactions on Evolutionary Computation*, 2025.

# Thank You